Bullseye Agent

Installation & Configuration Guide

v1.0.11

CONTENTS

1. Overview
2. Prerequisites
3. Download
4. Configure
5. Provider Setup
6. Install with Docker (Recommended)
7. Install with Python
8. Verify the Connection
9. Configuration Reference
10. Troubleshooting
11. Security
12. Support

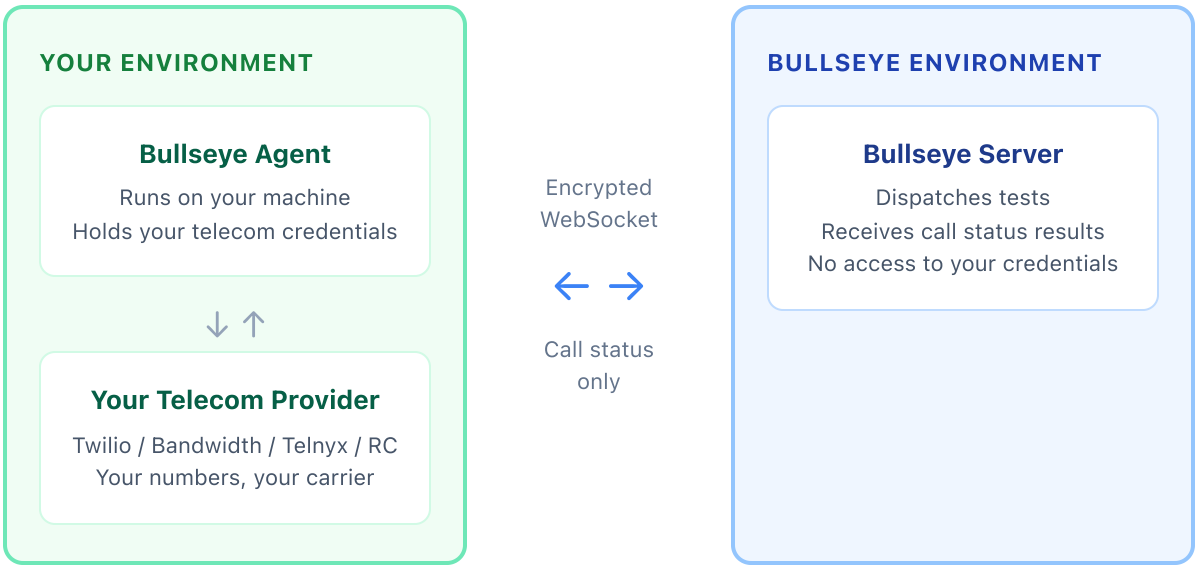
1. Overview

The Bullseye agent is a lightweight service that runs on your infrastructure. When it receives a test instruction from the Bullseye server, it places a real phone call through your telecom provider and reports the result.

Calls originate from **your own phone numbers** on **your own carrier**, giving an accurate picture of whether those numbers are being flagged as spam.

HOW IT WORKS

The agent maintains a persistent, encrypted WebSocket connection to the Bullseye server. When a test is queued, the server pushes the instruction to the agent in real time. The agent dials the call, monitors its progress, and reports the outcome (answered, no answer, busy, or failed) back to the server. Your telecom credentials never leave your machine.



2. Prerequisites

Before you begin, make sure you have:

Item	Details
Bullseye credentials	Your Bullseye contact will provide a **server URL** and an **API key** (starts with `bse_`).

Telecom provider account	**Bandwidth** — Account ID, OAuth Client ID/Secret, Application ID **Twilio** — Account SID + primary Auth Token **Telnyx** — API key + Call Control Connection ID **RingCentral** — Client ID, Client Secret, JWT token + Node.js 18+ **Asterisk** — ARI URL + credentials + a dial endpoint template **FreeSWITCH** — ESL host + password + a dial endpoint template **Proprietary** — your own network code in `providers/proprietary_provider.py`
Server / VM	Any Linux, macOS, or Windows machine with outbound internet access. Docker is recommended; Python 3.10+ is the alternative.
Network access	Outbound HTTPS (port 443) to the Bullseye server and your provider's API (e.g. `api.twilio.com`). No inbound ports required.

3. Download

Download the latest agent package:

↓ bullseye-agent-1.0.11.tar.gz ↓ bullseye-agent-1.0.11.zip

Extract the archive:

```
# Linux / macOS
tar xzf bullseye-agent-1.0.11.tar.gz
cd armor-bullseye-client-app-1.0.11

# Windows (PowerShell)
Expand-Archive bullseye-agent-1.0.11.zip -DestinationPath .
cd armor-bullseye-client-app-1.0.11
```

You should see the following files:

```
bullseye-agent/
  main.py               # Agent entry point
  requirements.txt      # Python dependencies
  providers/            # Telephony provider integrations
  Dockerfile            # Docker image definition
  docker-compose.yml    # Docker Compose service
  .env.example          # Configuration template
```

4. Configure

Create your configuration file from the provided template:

```
cp .env.example .env
chmod 600 .env
```

Open `.env` in a text editor and fill in the required values:

Variable	Required	Description
`BULLSEYE_SERVER_URL`	Yes	Server URL provided by Bullseye (e.g. `https://your-bullseye-server`)
`BULLSEYE_API_KEY`	Yes	Your agent's API key (starts with `bse_`)
`TELEPHONY_PROVIDER`	Yes	`bandwidth`, `twilio`, `telnyx`, `ringcentral`, `asterisk`, `freeswitch`, or `proprietary`

Then set the credentials for your provider. See the next section for details.

5. Provider Setup

Configure the credentials for your telephony provider. Only one provider is needed.

Bandwidth

Bandwidth is a CPaaS platform with direct programmatic voice. Calls are one-legged and fully automated. Bandwidth is the underlying carrier for many other providers including RingCentral.

What you need:

- A Bandwidth account with Voice API enabled. A free Bandwidth Build account works for evaluation; trial accounts can only call the verified mobile number on file until a credit card is added.
- **OAuth API credentials** (Client ID + Client Secret) created in the Bandwidth dashboard: **Account → API Credentials → Create**
- A Voice Application (provides the Application ID)
- A phone number assigned to that application

Variable	Description
`BANDWIDTH_ACCOUNT_ID`	Your Bandwidth account ID (numeric)
`BANDWIDTH_CLIENT_ID`	OAuth Client ID
`BANDWIDTH_CLIENT_SECRET`	OAuth Client Secret
`BANDWIDTH_APPLICATION_ID`	Voice application ID
`BANDWIDTH_ANSWER_URL`	Pre-filled: `https://your-bullseye-server/bandwidth/answer`

LEGACY BASIC AUTH DEPRECATED

Bandwidth's legacy username/password authentication is still accepted by the agent for backwards compatibility (`BANDWIDTH_API_USERNAME` and `BANDWIDTH_API_PASSWORD`) but is being decommissioned by Bandwidth on **2026-12-02**. New accounts should use OAuth.

ABOUT THE ANSWER URL

Bandwidth requires a webhook URL for call control. The `BANDWIDTH_ANSWER_URL` points to a Bullseye server endpoint that tells Bandwidth to hold the call for a randomized 35-55 seconds (varied per call so Bullseye traffic doesn't look like automation) and hang up. This is pre-configured in `.env.example` — you don't need to change it.

Additional requirements: None. Works with Docker or Python.

Twilio

Twilio is a CPaaS platform with direct programmatic voice. Calls are one-legged and fully automated.

What you need:

- A Twilio account
- The primary Account SID and Auth Token from the Twilio Console dashboard (not an API key)
- A phone number with voice capability purchased in the account

Variable	Description
`TWILIO_ACCOUNT_SID`	Account SID (starts with `AC`)
`TWILIO_AUTH_TOKEN`	Primary Auth Token from the Console dashboard

> **USE THE AUTH TOKEN, NOT AN API KEY**
>
> Twilio API keys (`SK...`) have restricted permissions and will not work. Use the primary Auth Token displayed on the Console dashboard.

Additional requirements: None. Works with Docker or Python.

Telnyx

Telnyx is a CPaaS platform with Call Control APIs. Calls are one-legged and fully automated. Note that call events have a 15–20 second delay in the Telnyx API; this does not affect call completion.

What you need:

- A Telnyx account with a **Voice API (Call Control)** application — not TeXML, not SIP Connection
- An API key from the Telnyx Mission Control portal
- A phone number assigned to the Call Control application
- An outbound voice profile assigned to the application

Variable	Description
`TELNYX_API_KEY`	API key from the Telnyx portal
`TELNYX_CONNECTION_ID`	Call Control application ID

Additional requirements: None. Works with Docker or Python.

RingCentral

RingCentral is a UCaaS platform. Unlike Bandwidth, Twilio, and Telnyx, it does not support simple REST-based outbound calling. The Bullseye agent uses a **SIP sidecar** — a small Node.js service that registers as a softphone on your RingCentral account and places calls directly via SIP.

What you need:

- A RingCentral account with a REST API application registered at developers.ringcentral.com
- A JWT credential generated in the RingCentral admin portal
- An **"Other Phone" device** on the extension. To create: Admin Portal → Phones & Devices → Add Device → Existing Phone.

Variable	Description
`RINGCENTRAL_CLIENT_ID`	REST API app Client ID
`RINGCENTRAL_CLIENT_SECRET`	REST API app Client Secret
`RINGCENTRAL_JWT_TOKEN`	JWT credential from the admin portal
`RINGCENTRAL_SERVER_URL`	Optional. Defaults to `https://platform.ringcentral.com`
`SIDECAR_AUTH_TOKEN`	Shared secret between the agent and the sidecar. **Required for the Docker profile** (the sidecar listens on the Compose network, not loopback, and refuses to start without it). Generate with `openssl rand -hex 32`. Optional for Python-from-source.

SIP SIDECAR

RingCentral requires a SIP sidecar to run alongside the agent:

Docker: set `SIDECAR_AUTH_TOKEN` in `.env` first, then start with the `ringcentral` profile: `docker compose --profile ringcentral up -d --build`. The sidecar runs as a second container and requires the shared token, which both containers read from `.env`.

Python: requires **Node.js 18+** installed on the machine. The agent detects this and starts the sidecar as a subprocess on `127.0.0.1` automatically. The first run will install npm dependencies. `SIDECAR_AUTH_TOKEN` is optional in this mode.

No public IP or inbound ports are needed — SIP runs over an outbound WebSocket connection.

CALLER ID LIMITATION

The “from” number is determined by which RingCentral device the sidecar registers as. You cannot specify a different caller ID per call.

Asterisk

Use this if you run your own Asterisk PBX (13.1 or newer) and want to originate calls through it. The agent talks to Asterisk via ARI (Asterisk REST Interface) over HTTP + WebSocket.

What you need:

- An Asterisk instance reachable from the agent host
- ARI enabled (`ari.conf`) with a user the agent can authenticate as
- HTTP server enabled (`http.conf`) — usually port 8088
- A dialplan landing-pad context for the call to enter after the originate (see below — each call is held open for a randomized 35-55 seconds)
- An outbound SIP trunk or endpoint configured for the destinations you want to dial

Variable	Description
`ASTERISK_ARI_URL`	ARI base URL, e.g. `http://asterisk-host:8088/ari`
`ASTERISK_ARI_USERNAME`	ARI user from `ari.conf`
`ASTERISK_ARI_PASSWORD`	ARI password from `ari.conf`
`ASTERISK_ENDPOINT_TEMPLATE`	Dial endpoint template, e.g. `PJSIP/{to_number}@my-trunk`
`ASTERISK_CONTEXT`	Dialplan context the originated call enters (e.g. `bullseye-landing`)
`ASTERISK_EXTENSION`	Optional. Extension within that context (default `s`)

`ASTERISK_DIAL_TIMEOUT`	Optional. Dial timeout in seconds (default `30`)

> **SBC DEPLOYMENTS**
>
> If you want Bullseye to dial through your own SBC, the `sbc-asterisk/` directory in the agent repo ships a docker-compose layout that runs both the agent and an Asterisk sidecar configured against your SBC. See `sbc-asterisk/README.md`.

Additional requirements: network reachability to your Asterisk ARI HTTP port (8088 by default).

FreeSWITCH

Use this if you run your own FreeSWITCH instance. The agent connects to FreeSWITCH's Event Socket (ESL), originates the call, and watches the event stream for the terminal hangup cause.

What you need:

- A FreeSWITCH instance with `mod_event_socket` loaded (loaded by default in vanilla FreeSWITCH)
- ESL bound on an interface reachable from the agent host, with a strong password configured in `event_socket.conf.xml` (**change the default `ClueCon`**)
- A SIP profile or gateway configured for outbound calls to the destinations you want to dial

Variable	Description
`FREESWITCH_HOST`	ESL host or IP
`FREESWITCH_PORT`	Optional. ESL port (default `8021`)
`FREESWITCH_PASSWORD`	ESL password from `event_socket.conf.xml`
`FREESWITCH_ENDPOINT_TEMPLATE`	Dial string template, e.g. `sofia/gateway/my-provider/{to_number}`

`FREESWITCH_DIAL_TIMEOUT`	Optional. Total seconds to wait for the terminal event (default `180`)

> **ESL IS UNENCRYPTED**
>
> The FreeSWITCH event socket is plain TCP, no TLS. Keep the agent and FreeSWITCH on a private network, or tunnel the ESL connection (Tailscale / WireGuard / SSH tunnel).

Additional requirements: network reachability to your FreeSWITCH ESL port (8021 by default).

Proprietary (your own network)

Use this if you originate calls through your own telecom infrastructure — one that isn't a hosted CPaaS, Asterisk, or FreeSWITCH. You implement a single Python function (`place_call`) that calls into your internal system, and the agent handles everything else (WebSocket auth, reconnect, event streaming).

What you need:

- A way to programmatically place a call on your network (an internal SDK, REST API, message queue, etc.)
- Whatever credentials / configuration your network needs — you can add them as environment variables that your `place_call` reads

See `CUSTOM_PROVIDER_GUIDE.md` in the agent repo for the full walkthrough: function signature, expected return values, status codes, and a reference implementation sketch.

The stub lives at `providers/proprietary_provider.py` — open it, find the `TODO` block inside `place_call`, and fill it in with your network call. No other code changes required.

Additional requirements: none from Bullseye's side. Your implementation determines what your environment needs.

6. Install with Docker (Recommended)

Docker handles Python, dependencies, and automatic restarts.

1 Install Docker

If Docker is not already installed, follow the official instructions for your operating system at docs.docker.com/get-docker.

2 Start the agent

```
docker compose up -d --build
```

The agent will build, start, connect to the Bullseye server, and automatically restart if the machine reboots or the process crashes.

RingCentral users: start with the SIP sidecar profile instead:

```
docker compose --profile ringcentral up -d --build
```

3 Check the logs

```
docker logs -f bullseye-agent
```

You should see output similar to:

```
        OOOOOOOOOO
      OO          OO
    OO   OOOOOOOO   OO
   OO  OO        OO  OO
  OO  OO   OOOO   OO  OO
  OO  OO  OO  OO  OO  OO
  OO  OO  OO * OO  OO  OO
```

```
   oo  oo  oo   oo  oo  oo
   oo  oo   ooooo   oo  oo
    oo  oo         oo  oo
     oo   ooooooooo   oo
       oo           oo
         ooooooooooo

       B U L L S E Y E  v1.0.11

12:00:01 INFO     Server:   configured
12:00:01 INFO     Provider: twilio
12:00:01 INFO     Connecting to server...
12:00:02 INFO     ============================================================
12:00:02 INFO     CONNECTED — agent is ready to receive tests
12:00:02 INFO     ============================================================
```

Press `Ctrl+C` to stop tailing (the agent continues running in the background).

> **YOU'RE DONE**
>
> The agent is running and connected. It will receive and execute test calls automatically. No further action is needed.

Common Docker commands

Command	Description
`docker compose up -d --build`	Start or rebuild the agent
`docker compose down`	Stop the agent
`docker compose restart`	Restart (picks up `.env` changes)
`docker logs -f bullseye-agent`	Follow logs in real time
`docker logs --tail 100 bullseye-agent`	Show last 100 log lines

7. Install with Python (Alternative)

Use this method if Docker is not available on your machine.

1 Verify Python version

```
python3 --version    # must be 3.10 or newer
```

2 Create a virtual environment and install dependencies

```
python3 -m venv .venv
source .venv/bin/activate       # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

3 Start the agent

```
python3 main.py
```

KEEP IT RUNNING

When running from source, use a process supervisor (`systemd` , `launchd` , `supervisord`) so the agent restarts automatically after reboots or crashes. See the README for a sample `systemd` unit file.

8. Verify the Connection

Once started, the agent should show:

```
CONNECTED — agent is ready to receive tests
```

If you see this message, the agent has successfully authenticated with the Bullseye server and is waiting for test instructions. No further configuration is needed.

To confirm a test call works end-to-end, contact your Bullseye representative and ask them to queue a test for your agent. You will see the call progress in the logs:

```
20:11:58 INFO     Test abc-123: +15555550100 -> +15555550199
20:11:58 INFO     Dialing +15555550100 -> +15555550199
20:11:58 INFO     Call initiated: sid=CA1234567890abcdef
20:12:12 INFO     Test abc-123: status=answered duration=14.5s
20:12:13 INFO     Test abc-123: result sent (answered)
```

9. Configuration Reference

All configuration is done through the `.env` file.

Shutdown behaviour

When the agent is told to stop — `docker compose down`, a Kubernetes pod termination, or Ctrl-C — it shuts down gracefully: it stops accepting new tests, waits for the calls already in flight to finish, delivers their results to the Bullseye server, then exits. A second stop signal exits immediately.

Variable	Description
`BULLSEYE_SHUTDOWN_TIMEOUT`	Optional. Seconds to wait for in-flight calls on shutdown (default `120`). A test call runs roughly

	60-90 seconds, so the default allows one to complete.

RUNNING ON KUBERNETES? SET THE GRACE PERIOD

Set the pod's `terminationGracePeriodSeconds` to `BULLSEYE_SHUTDOWN_TIMEOUT` **plus about 30 seconds** — Kubernetes defaults it to only 30 in total. The extra allowance covers waiting for the server to acknowledge the drained results and closing the connection:

```
spec:
  template:
    spec:
      terminationGracePeriodSeconds: 150   # 120s timeout + 30s
```

The agent can only finish its in-flight calls if the kubelet gives it the time. With the default grace period, every deploy, scale-down or node drain force-kills the agent part-way through and the results of all calls in flight are lost — those tests are reported as failed even though the calls themselves may have connected normally.

Log verbosity

Set `LOG_LEVEL` to control how much the agent logs:

Value	Output
`INFO` (default)	Connection status, test start and result
`DEBUG`	Everything, including per-poll provider status updates
`WARNING`	Only errors, reconnections, and unexpected events

Insecure mode (local testing only)

The agent refuses to connect to `http://` URLs by default because the API key would be transmitted in cleartext. To override for local development, set:

```
BULLSEYE_ALLOW_INSECURE=1
```

Do not use this in production.

10. Troubleshooting

Bundled network diagnostic: `net-check`

Every agent image ships with a diagnostic script that probes the network path from the agent container to the Bullseye server and to the configured telephony provider. Run it from inside a running container to distinguish credential problems from network problems.

Docker:

```
docker exec bullseye-agent net-check
```

Kubernetes / EKS:

```
kubectl exec -it deployment/bullseye-agent -- net-check
```

The output is a PASS / FAIL per stage (DNS, TCP, TLS, HTTP) for each endpoint the agent needs. It does **not** include credentials — safe to paste into a support ticket.

Result	Likely cause
DNS FAIL	CoreDNS / cluster DNS policy blocking lookups
TCP FAIL	Firewall, security group, or NetworkPolicy dropping the SYN
TLS FAIL	TLS-inspecting proxy or clock skew (cert validation)

HTTP FAIL with “connection reset by peer”	Deep-packet-inspection appliance dropping the request mid-stream
All checks PASS but calls still fail	Rule out network — likely credentials or provider config

The container also has `curl`, `dig`, `nslookup`, `ping`, and `nc` available if you want to probe something specific.

"Invalid API key" or connection closes with code 4003

Double-check `BULLSEYE_API_KEY` in your `.env` file. It should start with `bse_`. Contact your Bullseye representative if you believe the key is correct.

Agent keeps reconnecting

- Verify the server is reachable: `curl -v $BULLSEYE_SERVER_URL/docs`
- Confirm the URL starts with `https://`
- Check that outbound port 443 is not blocked by a firewall or proxy

"Error: Unsupported provider"

`TELEPHONY_PROVIDER` must be exactly one of `bandwidth`, `twilio`, `telnyx`, `ringcentral`, `asterisk`, `freeswitch`, or `proprietary` (lowercase, no quotes).

Twilio: authentication errors

Use the primary **Account SID** and **Auth Token** from the Twilio Console dashboard, not an API key (`SK...`). API keys have restricted permissions that prevent call placement.

Bandwidth: calls fail immediately

Verify your Bandwidth Voice Application has the correct Application ID and that `BANDWIDTH_ANSWER_URL` in `.env` points to the Bullseye server endpoint (`https://your-bullseye-server/bandwidth/answer`).

RingCentral: sidecar won't start

- **Docker:** make sure you used `--profile ringcentral`
- **Python:** verify Node.js 18+ is installed (`node --version`)
- Check that an “Other Phone” device exists on the RingCentral extension

RingCentral: "No OtherPhone device found"

Create one in the RingCentral Admin Portal → Phones & Devices → Add Device → Existing Phone.

Calls are not being placed

Check the logs for error messages after the test arrives. Common causes:

- The "from" number is not assigned to your provider account
- The account does not have outbound voice capability enabled
- Provider credentials are invalid or expired

11. Security

What is transmitted to the Bullseye server

The agent communicates **only call status information** to the Bullseye server:

- Call progress events: `dialing` , `answered`
- Final call result: `answered` , `no_answer` , `busy` , or `failed`
- Call duration (in seconds)
- The provider's call ID (e.g. a Twilio Call SID)
- A generic error message if the call failed (e.g. “Call initiation failed”)

That's it. The Bullseye server receives just enough information to know whether the phone number was reachable. No call data is stored on the Bullseye server or the test devices beyond the duration of the test. The phone numbers used to initiate and receive test calls are processed transiently to execute the test and are not retained.

What stays on your machine

SECURITY & PRIVACY

Bullseye runs entirely on your infrastructure and uses your existing telecom credentials to place test calls. Your API keys, tokens, and account IDs never leave your environment. The only information sent to ARMOR® is the call status (e.g., spam label, blocked, delivered) and the phone number being tested. No call content or recordings are ever collected. All connections are encrypted with TLS and Armor Solutions is SOC 2 Type II compliant. You can remove Bullseye from your environment at any time.

Encryption

All communication between the agent and the Bullseye server uses TLS-encrypted WebSocket connections (`wss://`). The agent refuses to connect over unencrypted channels by default.

Agent isolation

Each agent authenticates with a unique API key and only receives tests addressed to its own agent ID. Agents cannot see other agents' tests or access any server-side data.

Protecting the configuration file

- Restrict file permissions: `chmod 600 .env`
- Never commit `.env` to version control (excluded by `.gitignore` and `.dockerignore` by default)
- Each machine running an agent should have its own `.env`

Limiting exposure

To bound the impact if the Bullseye server or API key were compromised:

- Use a dedicated provider subaccount for Bullseye testing
- Set monthly spend caps in your provider's dashboard
- Restrict international dialing if you only test domestic numbers

Log retention

Logs include phone numbers, which may be PII under GDPR, CCPA, or similar regulations. Docker log rotation is pre-configured at 10 MB × 5 files. Adjust in `docker-compose.yml` if your retention policy differs.

---

12. Support

If you encounter issues, contact your Bullseye representative with:

- Agent version (displayed at startup)
- The last 50 lines of logs (`docker logs --tail 50 bullseye-agent`)
- The first 12 characters of your API key (e.g. `bse_a1b2c3d4`)
